

CSCE 678 DISTRIBUTED SYSTEMS AND CLOUD COMPUTING · MODULE 04 · LECTURE NOTES

Replication and Consistency

Several copies, and what you promise about them.

MODULE OUTCOMES

- Compare replication strategies on their failure behaviour.
- Define linearizability and distinguish it from serialisability.
- Place the consistency models in order of strength.
- Explain the session guarantees and why they are enough.
- Choose a consistency model deliberately.

1 Replication strategies

Strategy	How it works	Failure behaviour
Single leader	All writes go to one designated node, which replicates them to followers synchronously or asynchronously.	Simple, and it eliminates write conflicts entirely — there is one order because there is one writer. The leader is a throughput bottleneck and a single point of failure , so failover is required, and failover is where the bugs are (Module 06).
Multi-leader	Several nodes accept writes and replicate to each other, reconciling afterwards.	Write conflicts are now your problem (Module 03 §4), and they are the expensive problem. Justified across datacentres or for offline-capable clients, and rarely otherwise.
Leaderless (quorum)	The client writes to w replicas and reads from r , with no designated leader at all. Dynamo-style.	No failover to get wrong , since there is nothing to fail over. Requires read repair and anti-entropy to converge, and gives weaker guarantees than it appears to — see below.

$$w + r > n$$

Write to w replicas and read from r , out of n total. **The condition guarantees that any read set intersects any write set**, so a read is guaranteed to contact at least one replica that has the latest completed write. **Overlap is the entire mechanism.** With $n = 3$, $w = 2$, $r = 2$ the system tolerates one failure for both reads and writes, which is the usual choice; $w = n$ with $r = 1$ gives very fast reads and fragile writes, and the reverse gives the opposite.

QUORUMS ARE WEAKER THAN THEY LOOK

$w + r > n$ guarantees overlap, not linearizability. The read will see a replica holding the latest completed write, but there is no mechanism forcing all replicas to apply concurrent writes in the same order.

So a read can return a newer value and a subsequent read an older one, if the two reads happen to contact different subsets of replicas — a monotonic-reads violation (§3) arising directly from the quorum mechanism.

And a write that failed may have reached some replicas before failing. It is then neither applied nor not applied: it will persist if those replicas win a future read repair, and vanish otherwise. **The caller was told the write failed.**

Read repair and anti-entropy converge things eventually, and 'eventually' is carrying real weight in that sentence — it means 'if writes stop, which they do not'. **If you need linearizability, you need consensus** (Module 06), not a quorum. Dynamo-style stores are frequently deployed on the assumption that $w + r > n$ provides strong consistency, and it does not.

2 The consistency models

Model	What it guarantees	What it costs
Linearizable	The system behaves as though there were one copy, and the order respects real time.	Consensus, and at least one round trip to a quorum. Unavailable during a partition (Module 05).
Sequential	All nodes see operations in the same order, but that order need not match real time.	Cheaper than linearizable, and surprising to users — your own write can appear to happen after someone else's later one.
Causal	Causally related operations are seen in order by everyone; concurrent ones may differ.	Available during a partition , and it is the strongest model of which that is true — which is why so much research went into it.
Eventual	If writes stop, replicas converge. Nothing is promised while they do not.	The cheapest, and it says almost nothing — 'returns the value 42 always' satisfies eventual consistency until a write arrives.
Read-your-writes	You observe your own writes.	A session guarantee , scoped to one client, not a global property. Cheap (§3).
Monotonic reads	Successive reads by one client never go backwards in time.	Also session-scoped, also cheap, and it removes one of the two anomalies users actually report.

LINEARIZABILITY: THE SYSTEM BEHAVES LIKE ONE COPY

Every operation appears to take effect at a single instant between its invocation and its response, and the resulting total order is consistent with real time.

So if client A's write completes before client B's read begins, B sees it — even on a different machine, even hitting a different replica, with no coordination between A and B. That real-time requirement is what makes it the strongest practical model and what makes it expensive.

It is a *recency* guarantee about single objects. Serialisability is an *isolation* guarantee about multi-object transactions (CSCE 608 Module 08) — that some serial order of transactions produces the same result. **They are different and orthogonal properties**, routinely confused because both are informally called 'strong consistency'. A system can be serialisable and not linearizable (transactions may be ordered in a way that contradicts real time), and combining both is called strict serialisability.

The cost is at least one round trip to a quorum on every operation — you cannot answer from a local replica, because you cannot know whether a newer write exists elsewhere. **Which is exactly why it becomes unavailable during a partition** (Module 05).

3 Session guarantees

MOST USER-VISIBLE ANOMALIES ARE SESSION ANOMALIES

'I posted a comment and then it vanished' is a read-your-writes violation. The write went to one replica; the subsequent page load was served by another that had not yet received it. The data is not lost and the user has no way to know that.

'The count showed 5, then 7, then 5 again' is a monotonic-reads violation. Two successive reads landed on replicas at different stages of replication, so time appeared to run backwards.

Both are fixable cheaply and locally. Route a user's reads to the replica that accepted their write for some interval; or have the client carry a version token and require the serving replica to have caught up to it before answering. Neither requires consensus or a quorum read.

So the system can remain eventually consistent globally while eliminating the anomalies people actually notice and complain about. That is usually a far better trade than global strong consistency, which costs a round trip on every operation to prevent anomalies that, in most applications, no single observer is positioned to detect. **Ask what an individual user can actually see,** rather than what the system as a whole guarantees.

4 Choosing

Operation	Needs	Why
Account balance, inventory decrement, seat booking	Linearizable.	Double-spending or double-booking is unacceptable , and the invariant is global.
Unique username registration	Linearizable.	Uniqueness is a global invariant — two replicas cannot both accept the same name.
Social feed, comment threads	Causal.	A reply must never appear before the comment it replies to , which is exactly a causal guarantee. Strong consistency would be overkill.
Like counts, view counts, metrics	Eventual.	Nobody can tell, and nobody cares. Approximate counters are frequently used deliberately.
Session and preference state	Read-your-writes.	Only that one user observes it, so a session guarantee is exactly the right scope (§3).
Game world state	A single authority.	Not a consistency model at all — see below.

Consistency is chosen per operation, not per system. Treating one database setting as covering an entire application is the usual mistake, and it leads either to paying for linearizability on like counters or to losing inventory.

A GAME SERVER SIDESTEPS THE WHOLE PROBLEM

There is exactly one authoritative copy of the world: the server's. Clients do not hold replicas — they hold *predictions*, which is a different thing with a different contract.

So there is no consistency model to choose. No quorum, no conflict resolution, no convergence criterion, no reconciliation between equals. **There is an authority, and there are guesses about what the authority will say.**

Clients are permitted to be wrong and are corrected (Module 10). That is a completely different bargain from replication, and it is why the netcode modules look so unlike the first half of this course despite solving a superficially similar problem.

It works because a game has a natural trusted party and a bounded world. And it stops working the moment you need two authoritative servers — at which point you are back to Module 06 with a hard real-time deadline attached, which is why sharding a seamless game world across servers is genuinely difficult and why most games do not.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Kleppmann — Designing Data-Intensive Applications, chapters 5, 6 and 9	The best treatment of this entire module , particularly the linearizability-versus-serialisability distinction of §2.
Herlihy & Wing — Linearizability: A Correctness Condition for Concurrent Objects (free)	The original definition. Worth reading for how carefully it is stated.
Terry et al. — Session Guarantees for Weakly Consistent Replicated Data (free)	The §3 result , and the paper that named read-your-writes and monotonic reads.
DeCandia et al. — Dynamo: Amazon's Highly Available Key-value Store (free)	The quorum system of §1, with the sibling-value conflict handling that LWW replaced in many later systems.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement single-leader replication with asynchronous followers.
- 2 **Kill the leader mid-write** and report what the followers contain.
- 3 Implement quorum reads and writes with configurable n , w , and r .

- 4 **Demonstrate a non-monotonic read** under $w + r > n$ by contacting different replica subsets.
- 5 **Demonstrate a partially-applied failed write** and explain why the caller cannot know.
- 6 Run a linearizability checker (Porcupine or Knossos) over a history from your quorum store. Report the result.
- 7 Implement read-your-writes with a version token and verify the anomaly is gone.
- 8 Implement monotonic reads and verify the same.
- 9 **Take an application you know and classify each operation** by the consistency it actually needs.
- 10 **Measure the latency cost** of linearizable reads against local reads at several RTTs.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Compare three replication strategies on failure behaviour.
- 2 State the quorum condition and explain the overlap argument.
- 3 Give three ways quorums are weaker than they look.
- 4 Define linearizability, and distinguish it from serialisability.
- 5 Order six consistency models by strength.
- 6 Why is causal consistency significant?
- 7 Name two session anomalies and the cheap fix for each.
- 8 Why is consistency per operation rather than per system?
- 9 Why does a game server not need a consistency model?